

ConvLSTM for Action Recognition : CS 7643

Brinthakumari Rajeindram

brajeindram3@gatech.edu

Chi Zhao

czhao3336@gatech.edu

Gamini Singh

gsingh85@gatech.edu

Kyla Tracy

ktracy8@gatech.edu

Abstract

There is a growing demand for intelligent real-time systems to recognize fine-grained actions with accuracy. Traditional methods, while effective for general action recognition, often struggle with fine-grained tasks due to high computational costs and limitations in capturing both spatial and temporal features. In addition, a robust action prediction will serve as a catalyst for significant advancements in human-computer interaction in areas such as healthcare and sports analytics. This project explores the use of ConvLSTM, a hybrid model combining convolutional networks for spatial feature extraction, and LSTM for temporal sequence modeling. Specifically, we focus on the 50Salads dataset, which includes detailed annotations of salad preparation actions, to evaluate the performance of our approach. This report outlines our objectives, the experimental setup and results, loss functions, and benchmarked its accuracy scores against other state-of-the-art methods.

1. Introduction

Human Action Recognition (HAR) is a popular area of research for the use of deep learning architectures to identify intricate patterns and features from the spatio-temporal data of videos. Accurate HAR, performed on either individual or a group of people, is the basis for several computer vision and pattern recognition applications such as healthcare monitoring, surveillance, and sports performance analytics. Common traps encountered while building a robust HAR model include: technical limitations (variability in lighting, camera angles, scale, and occlusion impacts recognition accuracy), dataset limitations (lack of detailed, domain-specific annotations hinders model training and generalization) and operational limitations (reliable and real-time decision-making is difficult to achieve) [1].

1.1. Literature Review and Related Work

Skeleton-based methods such as Graph Convolutional Networks (GCNs) have shown promising results in HAR due to their compactness and their robustness to environmental variations. GCNs rely on predefined skeleton structure, which doesn't effectively handle different types of connection between joints. This makes it particularly challenging for them to work with raw video data. Additionally, GCNs separate spatial and temporal modeling, missing the opportunity to capture spatio-temporal relationships crucial for understanding actions [2, 3].

Diffusion models have also been used for temporal action segmentation of continuous frames in a video stream. Their focus on iterative refinement and reconstruction progressively adds/removes noise and makes it difficult to get precise action boundaries. Moreover, computational delays from extensive forward and reverse computations create bottlenecks in real-world scenarios [4, 5].

Such complex models add marginal gains at the cost of increased resource demands, and reduced practical applicability. ConvLSTM, though simpler, strikes a balance between performance and feasibility, making it a strong option for real-world applications like precipitation forecasting and video frame prediction. ConvLSTM combines Convolutional Neural Network (CNN) for extracting spatial features and Long Short-Term Memory Model (LSTM) for temporal sequence modeling [6, 7]. Research has shown that high dimensionality of the spatiotemporal sequences impacts ConvLSTM performance on intricate tasks, such as recognizing subtle variations in human actions (in fact this limitation forced us to reduce our prediction classes, as discussed in later sections) [8].

Further studies have been conducted to enhance ConvLSTM for more accurate and context-specific predictions. Examples include: augmenting ConvLSTM architecture with self-attention for better feature selection [9], integrating ConvLSTM as the generator in a Generative Adversarial Network (GAN) for realism [10], and implementing a ConvLSTM-based autoencoder architecture for more accu-

rate localized forecasting [11].

1.2. Objective

Considering the time and computational resource constraints, we prioritized simplicity and practicality over complexity. We applied ConvLSTM to build a system that can identify detailed actions in video sequences of salad preparation. Specifically, we focused on breaking down video data into meaningful stages, making machines capable of understanding and labeling these actions efficiently and accurately. If successful, our work could significantly advance human-computer interaction in wide variety of areas, ex: accurate monitoring of patient activities, getting a deeper understanding player movements for training. Beyond these domains, our system could make smart environments, where machines are enabled to better understand human activities in a more intuitive way.

To achieve our objective, we used the 50Salads dataset from the University of Dundee [12]¹. The dataset consists of 50 videos of 25 participants preparing two types of salads, encompassing over 500,000 frames (10,000 frames each). Each frame is annotated with timestamps and corresponding action labels, providing a rich resource for fine-grained action recognition. 5 of the 50 videos contained corrupted pixels (14-1,16-2,17-1,18-1,18-2), and as such were removed from the dataset along with their corresponding timestamps and annotations. Before training, the action behaviors were encoded 1-56 (consolidated into 5 targets), and 4 of the action labels that overlapped with other labels were removed. This dataset has been used in 29 papers, mostly for action segmentation, and temporal convolutions [13]. Due to the dataset’s size, we employed a generator to load one video at a time, used a moving window to process and load frames one at a time into our model, and converted RGB videos into grayscale to ensure efficient use of computational resources.

The rich annotations and large-scale video data make 50Salads a reliable choice for benchmarking the performance of our ConvLSTM model. This level of detail allows the model to learn subtle variations in human actions. Moreover, the dataset’s diversity in participant behaviors and sequences ensures our findings are generalizable.

2. Approach

We focused on developing ConvLSTM (from scratch) by using PyTorch, tuning the model hyper-parameters, and documenting the successes and limitations of the network. We followed the architecture originally proposed by Shi et al. [6]. The authors publicly published code for their core

¹Note, part of the database that holds the data is now unable to be connected to giving only an 'Error Establishing a Database Connection' when accessed. We were able to download the original data before the site error, and has been stored in box [14, 15].

methodology, but we could not use it directly because it referenced Tensorflow and we have used PyTorch for our model development [16].

2.1. Model Architecture

In standard LSTMs, the gates (forget, input, output) and the cell state are computed through matrix multiplications between the input and weights, which requires the input data to be flattened into a vector. In contrast, ConvLSTM operates directly on the images, by performing convolutions on both the input and the previous hidden state with the respective weights. This enables ConvLSTM to extract and process spatial features from the input image or sequence of images in their original form. The equations of the gates, cell states and hidden states in ConvLSTM are illustrated and implemented as shown, and it can be noted the presence of the convolution operation as discussed.

Forget Gate:

$$f_t = \sigma(W_f * x_t + U_f * h_{t-1} + b_f)$$

Input Gate:

$$i_t = \sigma(W_i * x_t + U_i * h_{t-1} + b_i)$$

Cell State:

$$C_t = f_t * C_{t-1} + i_t * \tanh(W_C * x_t + U_C * h_{t-1} + b_C)$$

Output Gate:

$$o_t = \sigma(W_o * x_t + U_o * h_{t-1} + b_o)$$

Hidden State:

$$h_t = o_t * \tanh(C_t)$$

Figure 1. Convolution operation implemented in LSTM resulting in ConvLSTM

Our model architecture incorporates a ConvLSTM cell, which replaces the fully connected layers of traditional LSTM with 2-dimensional convolution operations to compute the gate values and update the hidden and cell states. This step is followed by iteratively passing each image frame of the video sequence through the ConvLSTM cell, enabling the model to process both the spatial and temporal dependencies across the sequence. As the ConvLSTM processes each frame at each time step, the hidden state will now capture both spatial and temporal information from the video sequence. Finally, fully connected linear layer and softmax function is also added to enable multi-class classification. The model architecture is illustrated in Figure 2.

ConvLSTM cell was designed to process spatiotemporal sequences, where the input was represented as a 5D tensor of shape (batch size, sequence length, out channels, height, width). Each sequence consisted of multiple video frames, with spatial dimensions (height and width) capturing the pixel-level information. Channel dimension was set to 1 due to grayscale videos. Sequence length is basically the temporal dimension. Depending on whether predictions

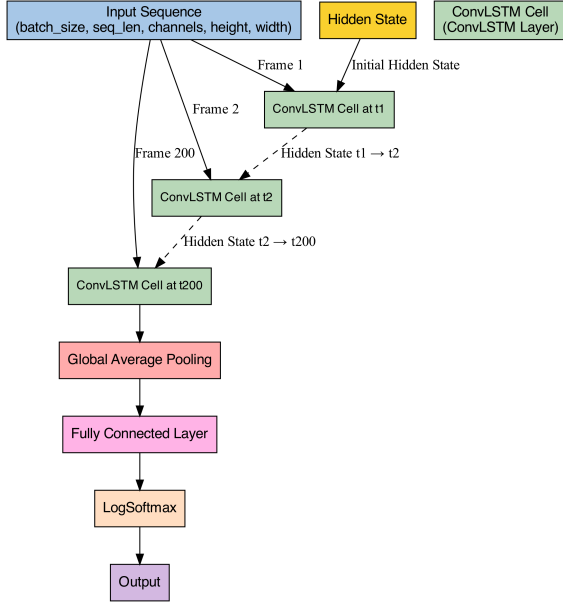


Figure 2. Model Architecture

were frame-wise or sequence-level, the output of ConvLSTM cell was either a 5D tensor of shape (batch size, sequence length, channels, height, width) or aggregated representations such as (batch size, channels, height, width). These outputs were passed to a fully connected layer and finally a softmax layer for action classification. This representation ensured compatibility with spatiotemporal features while maintaining computational efficiency.

To ensure the model’s robustness and understanding of important moments in the video, several techniques were employed. To investigate how well the network is learning during training, the gradients during the backward pass were visualized by registering hooks for the ConvLSTM cell. This helped in identifying potential issues like vanishing or exploding gradients. Gradient flow was monitored at the last epoch during the training step specifically for the ConvLSTM cell module, allowing for a better understanding of how spatial and temporal data is emphasised in the network. The image in Figure 3 illustrates the gradient during the backward pass captured for peeling activity during salad prep that illustrates that the model is able to capture spatial information pertinent to the activity; in this instance a peeler.

Overall, learned parameters in the model resided primarily in the convolutional layers (learning spatial feature representations from individual frames) and LSTM units (learning temporal dependencies across frame sequences). These layers were trained through back-propagation to minimize cross-entropy loss function, ensuring predictions are evaluated effectively across sequences and spatial grids. Data preprocessing and post-processing steps did not in-

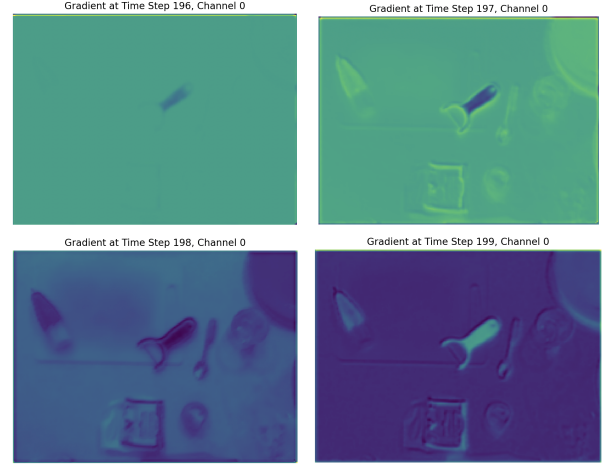


Figure 3. Gradient captured during backward pass for consecutive frames in ConvLSTM cells during model training for peeling activity in salad prep. The model is placing increasing emphasis on the peeler in successive time steps.

volve learned parameters. Using Softmax to normalize classifier probabilities (post-processing) was a deterministic operation that transformed raw predictions into actionable outputs without contributing to the model’s parameterized learning process.

2.2. Performance Issues and Optimization Efforts

Given the high dimensionality of the spatio-temporal sequences, we anticipated significant challenges with memory usage and managing the complexity of the ConvLSTM model.

The large number of frames in each video sequence was expected to cause memory bottlenecks. To address this, we proactively implemented a data generator to load one video at a time. Additionally, we used a moving window approach that processed smaller number of frames at a time. Frame augmentation techniques were also used to reduce unnecessary spatial data (resolution down scaling and extracting areas crucial to recognizing the action, such as areas with hands or salad ingredients). However, this attempt did not work as expected because the memory usage continued to remain high. The sequence’s size posed a persistent issue and memory leakage caused our kernel to die and/or our system to crash. As a result, we implemented deeper optimizations on CUDA memory management by deleting unused tensors, clearing processed frames and labels, and using in-place operations to overwrite existing tensors. These adjustments helped resolve memory issues, allowing the model to train effectively without exceeding system limits.

To further improve the model’s efficiency, we employed a variety of training based optimizations. Early stopping was used to avoid unnecessary epochs once the model converged. Learning Rate Scheduler was used to help the

model converge faster by varying learning rate to prevent the model from getting stuck in suboptimal local minima and wasting compute resources without significant performance lift. Finally, Alternate Precision Modeling was used to reduce memory consumption and speed up computations by utilizing 16-bit precision for model weights and computations (instead of the default 32-bit precision). We used Adam Optimizer because it combined the benefits of adaptive learning rates and momentum, making it particularly effective in traversing complex and uneven loss surface of spatio-temporal data with high dimensionality.

We unsuccessfully attempted to improve our model’s performance by implementing a self-attention mechanism [9] to capture subtle variations within individual frames and across frame sequences. Despite aforementioned optimization efforts, integrating attention was infeasible due to the available hardware constraints and resulted in consistent CUDA out-of-memory (OOM) errors. This issue stems from the self-attention module that has time and memory complexity quadratic in scale. Significant research has been conducted to reduce compute and memory requirements of transformers [19]. Future iterations of our work can target implementing these techniques.

3. Experiments

We used A100 GPU Hardware Accelerator in Google Colab for model training and validation (83.5GB System RAM and 40GB GPU RAM). Considering resource constraints, we processed 15 epochs for each attempt and used accuracy to evaluate performance on the validation set. To avoid overfitting and efficiently use compute resources, each attempt was set to suspend when the validation loss value stopped showing improvement in the last 5 epochs. We used greedy approach to tune model hyper-parameters in each attempt, which included experimenting with a range values for learning rate (1e-2, 5e-2, 1e-3, 1e-4 and 1e-5), kernel size (7x7, 3x3), hidden linear input dimensions (32, 64), and weight decay within Adam Optimizer to regularize our model (1e-4, 1e-5). For each iteration of our experiments, efforts were made to ensure that the selected hyper-parameter values work harmoniously together while maintaining stable and efficient training.

Additionally, we combined the original 56 behavior labels into 5 targets (“Add Ingredients”, “Mixing Ingredients”, “Cutting Ingredients”, “Placing Ingredients in Bowls”, and “Serving”). Figure 4 shows details on this process. For example, distinct behaviors for placing cucumber, tomato, lettuce and cheese into bowl were replaced with a common target “placing ingredients into bowl”. By aggregating the labels in this fashion, were we able to improve our overall run time and strike a balance between achieving high prediction accuracy and retaining enough granularity to make the predictions meaningful. This enabled the model

to perform better without compromising on the overall purpose of the task.



Figure 4. Consolidation of 56 Original Behaviors into 5 Target Labels. The figure includes behaviors that remained after cleaning redundant values.

We faced constraints in optimizing the batch size as a hyper-parameter due to significant memory requirements associated with loading each video. We were restricted to processing a single batch at a time. Some potentially risky strategies to mitigate this limitation include reducing the sequence length by sampling every x^{th} frame or minimizing the size and resolution of the images. We call them risky because reducing sequence length could lead to the loss of critical temporal information and reducing image resolution could remove spatial details essential for accurate action recognition. However, the potential compromise in data quality and model performance outweighed the benefits.

4. Results

4.1. Picking the "Winner"

We started our experimentation with 56 original behavior labels. Figure 5 shows the validation accuracy curves generated from 3 attempts of different hyper-parameter sets. Table 1 captures the corresponding hyper-parameter values.²

	LR	Kernel	Hd Dims	Wt Decay
Attempt 1	0.01	(7,7)	64	1e-4
Attempt 2	0.005	(3,3)	64	1e-4
Attempt 3	0.0001	(3,3)	64	1e-4

Table 1. Hyper-parameter Tuning Results on original 56 Behaviors. LR = Learning Rate, Hd Dims = Hidden Dimensions, Wt Decay = Weight Decay

- The instability of Attempt 1 suggests that the higher learning rate (0.01) might be causing the model to overshoot optimal parameters, resulting in unstable updates during training. The larger kernel size increased

²The whole hyperparameters tuning results can be viewed from [here](#).

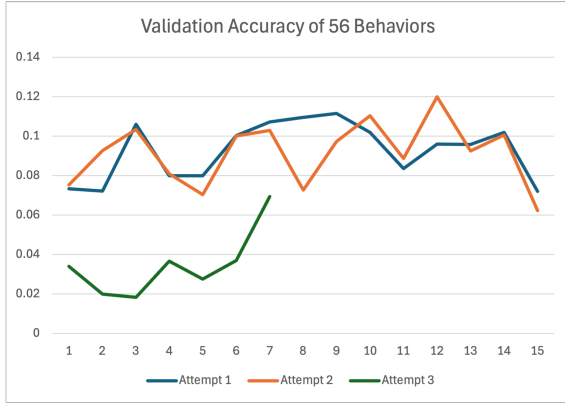


Figure 5. Validation Accuracy Curves using original 56 Behaviors. Curves appear across varying epoch lengths due to early stopping

spatial context but may have introduced noise, reducing model performance.

- Attempt 2 suggests that a smaller kernel size (3,3) and a reduced learning rate (0.005) stabilizes validation accuracy compared to Attempt 1 but the model is struggling to converge and the performance still oscillates.
- In Attempt 3, learning rate is further reduced (0.0001), but validation accuracy starts very low and shows a slow upward trend without significant improvement before early stopping terminates the run. This indicates underfitting due to insufficient parameter updates.

Across all our attempts, the highest observed accuracy was just 10%. These attempts gave us the reason to believe that model was facing extreme difficulties in distinguishing subtle differences between 56 behaviors. Therefore, we continued our experimentation with 5 consolidated target labels.

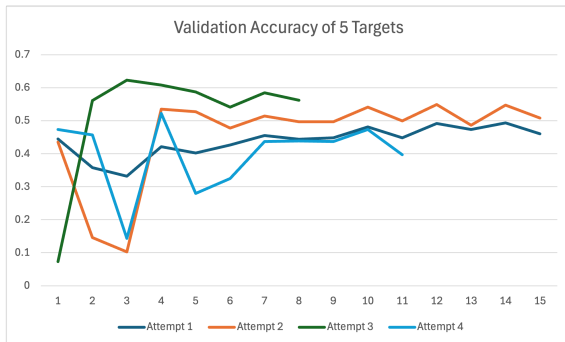


Figure 6. Validation Accuracy Curves using Consolidated 5 Target Values. Curves appear across varying epoch lengths due to early stopping

	LR	Kernel	Hd Dims	Wt Decay
Attempt 1	0.001	(3,3)	32	1e-4
Attempt 2	0.001	(3,3)	64	1e-4
Attempt 3	0.001	(3,3)	64	1e-5
Attempt 4	0.0001	(3,3)	64	1e-5

Table 2. Hyper-parameter Tuning Results on Consolidated 5 Target Values. LR = Learning Rate, Hd Dims = Hidden Dimensions, Wt Decay = Weight Decay

Figure 6 shows the validation accuracy curves generated from 4 attempts of different hyper-parameter sets. Table 2 captures the corresponding hyper-parameter values.³

- In Attempt 1, accuracy begins moderately at epoch 1 but drops sharply around epoch 3. It then climbs slowly and stabilizes at a lower level (40%) compared to other attempts. This is due to consolidated targets and using just 32 hidden dimensions, which compromises the spatio-temporal complexity needed to effectively classify.
- In Attempt 2, we increase the hidden dimensions to 64. Accuracy drops in the early epochs (around epoch 2–3) but climbs steadily and stabilizes around 50%. Increasing the hidden dimensions to 64 gives the model more capacity to learn spatio-temporal features, leading to better performance compared to Attempt 1.
- In Attempt 3, larger hidden dimensions (64) provided sufficient capacity for learning, while lower weight decay (1e-5) allowed the model to better adapt to the data without over-penalizing large weight values. This combination enabled faster convergence. Overall, this configuration demonstrated the best balance of learning rate, regularization, and capacity, achieving the highest validation accuracy (60%) and stability across all attempts.
- In Attempt 4, accuracy starts very low and climbs gradually, stabilizing around 45%. While the configuration avoided overfitting, the learning rate was too conservative, hindering the model's ability to reach optimal performance.

Since Attempt 3 emerged as our "winner" approach, it helps to pay a closer look at the training process for this model. Figure 7 shows the associated training and validation accuracy learning curves. The rapid rise in both curves during the initial epochs shows that the model quickly learns dominant patterns in the data. The stabilization of both curves after epoch 3 suggests that the model converged effectively. The combination of early stopping and regularization has prevented the model from overfitting the training

³The whole hyperparameters tuning results can be viewed from [here](#).

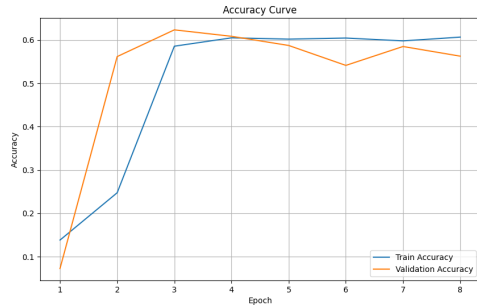


Figure 7. "Winner" Local Optimal Accuracy Learning Curve. LR=0.001, Kernel=(3,3), Hidden Dim=64, Weight Decay=1e-5, Target Values=5

data. Instead of prioritizing perfect training accuracy, the model focuses on patterns that generalize well to validation data, allowing the validation accuracy to remain higher than training after epoch 4.

4.2. Performance Benchmarking

Benchmarking our results allows us to evaluate the performance of our "winner" model against established standards (specific to this dataset and/or ConvLSTM architecture), ensuring its competitiveness and identifying areas for improvement.

Research studies based on the 50Salads dataset indicate that more complex and recent models have achieved over 80% accuracy, while using more target labels for recognition [17].⁴ It is important to consider nuances of model architectures for a fair comparison. For example, DiffAct model [5], which is ranked highly in terms of accuracy, introduces two critical characteristics, "position prior" and "relation prior". "Position prior" focuses on relationship between certain target behaviors and their time locations within the videos. "Relation prior" emphasizes the sequential order of certain target behaviors, which is crucial for modeling the logical progression of activities. These characteristics are not explicitly addressed by the standard ConvLSTM approach, leading to higher accuracy and robustness of DiffAct.

Research study involving simple ConvLSTM architecture was referenced as well for performance benchmarking [18]. This study uses a sports dataset that contains 200 videos with 50 to 51 targets. According to Fig. 4(a) of this paper, the validation accuracy for simple ConvLSTM reaches 90% after 20 epochs of training. However, the model initially struggles, showing validation accuracy well below 50% during the first 10 epochs. Furthermore, the validation curve exhibits overfitting signals, with a slight increase between epochs 5 and 10, indicating potential chal-

⁴Related researches can be found from [here](#).

lenges in generalizing early in the training process.

Given these insights, review of related studies suggests that ConvLSTM models have the potential to deliver strong performance when properly adapted to the 50Salads dataset. However, our current resources are insufficient to support an extensive and conclusive proof of this hypothesis.

5. Conclusion

In this project, we explored the combination of convolutional layers for spatial feature extraction and LSTM layers for temporal sequencing to recognize human actions during salad preparation. While our model faced challenges in distinguishing subtle variations between similar actions, it performed well in identifying broader categories of hand movements. Despite encountering memory constraints during training, our results indicate that ConvLSTM offers a strong alternative to more complex methods. While it may not capture every fine-grained detail, its scalability and ease of deployment position it as a valuable alternative to more computationally demanding approaches.

Through this project we also gained valuable insights into optimizing model hyper-parameters and managing cache memory to maximize performance within limited computational resources. On related note, research has shown the importance of simplifying models while maintaining strong performance in long-range spatiotemporal modeling. For instance, approaches like using single multi-function gates instead of multiple gates in ConvLSTM [20] or enabling parallel processing of spatio-temporal data fed into ConvLSTM [21] have shown promising results. Future work should focus on designing models that scale efficiently with sequence length and effectively capture long-range dependencies, while using ConvLSTM as its foundation. These models can be used for developing efficient systems in applications such as patient monitoring and player movement tracking, paving the way for further innovation in the field of human computer interaction.

References

- [1] Khater, et al. *A novel human activity recognition architecture: using residual inception ConvLSTM layer*
- [2] Li, et al. DD-GCN: Directed Diffusion Graph Convolutional Network for Skeleton-based Human Action Recognition
- [3] Huang, et al. Feature reconstruction graph convolutional network for skeleton-based action recognition
- [4] Wang, et al. Faster Diffusion Action Segmentation
- [5] Liu, et al. "Diffusion Action Segmentation"
- [6] Shi, Chen, et al. "Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting"
- [7] Demetrakopoulos, "Short-Term Precipitation Forecasting in the Netherlands: An Application of Convolutional LSTM Neural Networks to Weather Radar Data"

- [8] Yadav, Tiwari, et al. Skeleton-based human activity recognition using ConvLSTM and guided feature learning
- [9] Lin, Li, et al. Self-Attention ConvLSTM for Spatiotemporal Prediction
- [10] Ji, Gong, et al. CLGAN: a generative adversarial network (GAN)-based video prediction model for precipitation nowcasting
- [11] Patel, Tanwar, et al. FACT: Autoencoder and Attention Conv-LSTM- Based Collaborative Framework for Cloud Cover Prediction
- [12] Dundee University. 50 salads dataset. <https://cvip.computing.dundee.ac.uk/datasets/foodpreparation/50salads/>.
- [13] Papers with code. 50 salads dataset, papers. <https://paperswithcode.com/dataset/50-salads>.
- [14] Kyla Tracy. 50 annotations + timestamps. <https://gatech.box.com/s/bvpzhl3w453fb5ow3qg1qsy1rcwzy5i7>.
- [15] Kyla Tracy. 50 videos. <https://gatech.box.com/s/eq72vz4xgjtqwq0yc9m9bpdcgnzi792g>.
- [16] <https://github.com/petrosDemetrakopoulos/LSTM-radar-precipitation-forecast>
- [17] Action Segmentation on 50salads: <https://paperswithcode.com/sota/action-segmentation-on-50-salads-1>
- [18] Uddin, et al. Deep learning-based human activity recognition using CNN, ConvLSTM, and LRCN.
- [19] Dao, Fu, et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness
- [20] Elsayed, et al. Reduced-Gate Convolutional LSTM Using Predictive Coding for Spatiotemporal Prediction
- [21] Smith, Mello, et al. Convolutional State Space Models for Long-Range Spatiotemporal Modeling

6. Work Division

Student Name	Contributed Aspects	Details
Brinthakumari Rajeindram	Implementation, Gradient Hooks and Report Writing	Implemented ConvLSTM cell and ConvLSTM module in the code. Implemented registering of forward and backward hooks to capture input and gradients, and their visualization.
Chi Zhao	Loss and Accuracy Calculation, Hyperparameter Tuning, Result Analysis and Report Writing	Implemented Loss file, performed hyperparameter tuning, visualized learning curve comparison, analyzed results from aspects of dataset and model.
Gamini Singh	Optimization Techniques, Self-Attention Testing, Hyperparameter Tuning and Report Writing	Implemented memory and model optimization techniques, tested self-attention mechanism and performed hyperparameter tuning.
Kyla Tracy	Implementation, Data Loading, Generator, Moving Window and Report Writing	Implemented data loader to manipulate data into a usable state. Implemented main file, with generator and moving window for training memory management.

Table 3. Contributions of team members.